



Lab 1 – Evaluating AI Generated code Quality

Goal – learn to critically review AI output

Challenge Project: Task management API with complex business logic

Scenario: You're building a project management system where tasks have dependencies, priority levels, and resource allocation constraints.

AI Prompting Task: Use AI to generate a function that:

- Creates a new task with dependencies
- Validates task scheduling conflicts
- Calculates critical path for project completion
- Handles resource over-allocation scenarios

Specific Requirements:

Create a TaskManager class that can:

1. Add tasks with dependencies (prerequisite tasks)
2. Detect circular dependencies
3. Calculate earliest start dates
4. Assign resources without conflicts
5. Generate project timeline with critical path

Output: use the code review checklist (See next page). Annotate the code and prioritize issues



AI-Assisted Code Review Checklist

Correctness & Functionality

- ☐ Does the code compile/run without errors?
- ☐ Are all requirements or prompt instructions implemented?
- ☐ Are inputs validated and edge cases handled?
- ☐ Are assumptions explicit, or is the AI inferring behaviour without clear justification?
- ☐ Does the code produce the expected output in sample test cases?

Readability & Maintainability

- ☐ Is the code clear and self-explanatory?
- ☐ Are names (variables, functions, classes) meaningful and consistent?
- ☐ Is there unnecessary complexity or redundant logic (over-engineering)?
- ☐ Are comments concise, accurate, and useful?
- ☐ Does the style match the project's conventions (linting/formatting)?
- ☐ Could another developer easily understand and modify this code later?

Test Coverage

- ☐ Are there unit or integration tests for key functionality?
- ☐ Do tests check both success and failure paths?
- ☐ Are tests deterministic (no random, flaky behaviour)?
- ☐ Do coverage reports show adequate coverage (>80%)?
- ☐ Is test data realistic and not hard-coded in insecure ways?

Security

- ☐ Are all inputs validated and sanitized (especially user input)?
- ☐ Does the code avoid insecure functions or libraries?
- ☐ Are secrets, tokens, or credentials handled securely (not hard-coded or logged)?
- ☐ Does it correctly handle authentication and authorization where relevant?
- ☐ Are external calls (APIs, DB queries, file I/O) done safely?
- ☐ Have dependencies been scanned for known vulnerabilities?



Performance & Efficiency

- ☐ Is the algorithm appropriate for expected data volumes?
- ☐ Are there unnecessary loops, nested calls, or inefficient operations?
- ☐ Are external API or database calls minimized or batched when possible?
- ☐ Is memory or resource management handled properly?

Reliability & Robustness

- ☐ Does the code gracefully handle exceptions and unexpected states?
- ☐ Are fallback or recovery mechanisms implemented where needed?
- ☐ Are logging and error messages clear and actionable (no sensitive data in logs)?

Ethical & Legal Considerations

- ☐ Does the code include or reproduce any licensed or copyrighted material?
- ☐ Does it reference or copy large sections from training data examples?
- ☐ Is the AI-generated content clearly attributed or traceable if needed?
- ☐ Does the code comply with organizational or legal AI-use policies?

Documentation & Context

- ☐ Is there a clear explanation of what the code does and how to use it?
- ☐ Are AI-generated sections labeled or noted in comments/commits?
- ☐ Is rationale provided for non-obvious design choices?
- ☐ Are dependencies and setup steps documented?